

# 랜덤 포레스트 분류와 정확도의 함정

모델을 학습시키는 법부터 그 점수를 의심하는 법까지

RandomForest fit predict predict\_proba n\_estimators feature\_importances\_ accuracy\_score DummyClassifier

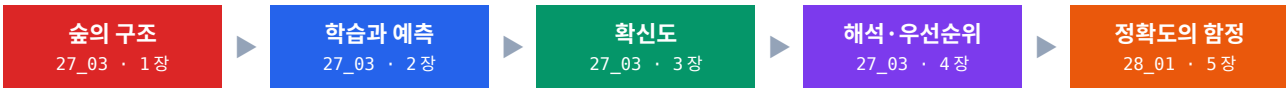
불균형

2026년 9월 11일 (금) · 38일차

교안 27\_03 랜덤 포레스트 고장 임박 분류 · 28\_01 혼동행렬 기반 분류 평가 (PART 01 정확도와 그 한계)

포스코 K-뉴딜 PDM 아카데미 · 박민호

## 오늘 다룬 것



| 오늘의 숫자          | 값                                | 무엇인가                                   |
|-----------------|----------------------------------|----------------------------------------|
| 엔진 데이터 분류 정확도   | 93.6%                            | 3,614 행 · 평가용 723 행 · n_estimators=100 |
| 가장 중요한 센서       | sensor_7 0.287 · sensor_15 0.200 | 둘이 합쳐 판단의 절반 가까이                       |
| 확신 있는 예측        | 확률 0.9 이상 39 건 중 35 건 적중         | 우선 점검 대상                               |
| 애매한 예측          | 확률 0.4~0.6 사이 38 건               | 한 번 더 지켜볼 대상                           |
| 1,000 행 데이터 정확도 | 모델 86.5% vs 더미 78.0%             | 차이 8.5%p — 정확도만 보면 안 되는 이유             |

## 연결

다음 시간 — 28\_01 PART 02 혼동행렬, PART 03 정밀도와 재현율. 오늘 정확도 하나로는 부족하다는 것까지 확인했으니, 다음은 네 칸짜리 성적표로 무엇을 놓쳤는지 갈라 봅니다.

오늘은 **드디어 모델을 학습시킨 날**입니다. 지난 시간까지 데이터를 준비해 두었으니, 오늘은 그 네 덩어리를 랜덤 포레스트에 넣고 예측을 받아 보았습니다. 그리고 수업 마지막에 방향이 한 번 꺾입니다 — 모델이 낸 점수를 그대로 믿어도 되는가 하는 질문으로요. 1~4 장이 모델을 만드는 이야기이고, 5 장이 그 점수를 의심하는 이야기입니다.

### 뱃지 — 이름 앞에 붙은 색 딱지

각 항목의 이름 왼쪽에 붙은 색 딱지는 그것이 문법적으로 무엇인지를 나타냅니다. 헛갈릴 때는 이렇게 판별합니다. **앞에 점(.)이 있나?** 없으면 함수나 키워드입니다. **점이 있으면 괄호()가 붙나?** 붙으면 메서드, 안 붙으면 속성입니다.

| 뱃지         | 뜻                                | 오늘 나오는 예                                                 |
|------------|----------------------------------|----------------------------------------------------------|
| <b>함수</b>  | 단독으로 호출하는 도구. 앞에 점이 없습니다.        | <code>accuracy_score() · np.zeros()</code>               |
| <b>메서드</b> | 값에 점을 찍고 괄호와 함께 부르는 도구.          | <code>.fit() · .predict() · .predict_proba()</code>      |
| <b>속성</b>  | 점을 찍되 괄호가 없습니다. 학습 결과로 채워진 값입니다. | <code>.feature_importances_</code>                       |
| <b>문법</b>  | 함수에 넘기는 옵션이나 표현 방식.              | <code>n_estimators=100 · strategy="most_frequent"</code> |
| <b>개념</b>  | 코드가 아니라 알고 있어야 할 생각의 틀.          | 다수결 · 베이스라인 · 정확도의 함정                                    |
| <b>패턴</b>  | 여러 도구를 엮어 쓰는 정형화된 코드 뭉치.         | 예측 · 실제 · 확률을 한 표로 모아 정렬                                 |

### 오늘 처음 나오는 도구

아래 도구들은 오늘 처음 등장합니다. 본문에서 쓰기 전에 모두 정식으로 소개하니, 코드를 읽다 막히면 이 표에서 몇 장으로 가면 되는지 확인하십시오.

| 도구                                      | 한 줄 역할                                 | 처음 나오는 곳 |
|-----------------------------------------|----------------------------------------|----------|
| <code>RandomForestClassifier</code>     | 트리 여러 그루의 다수결로 0/1 을 정하는 분류기입니다.       | 1 장      |
| <code>model.fit()</code>                | 학습용 입력과 정답을 보여 주며 모델 안에 규칙을 채웁니다.      | 2 장      |
| <code>model.predict()</code>            | 새 입력에 대해 0 또는 1 판단을 돌려줍니다.             | 2 장      |
| <code>model.predict_proba()</code>      | 0 일 확률과 1 일 확률을 돌려줍니다. 확신의 세기입니다.      | 3 장      |
| <code>model.feature_importances_</code> | 어떤 입력이 판단에 많이 쓰였는지 점수로 알려 주는 속성입니다.    | 4 장      |
| <code>accuracy_score()</code>           | 정답과 예측을 대조해 맞힌 비율을 계산합니다.              | 5 장      |
| <code>DummyClassifier</code>            | 아무것도 학습하지 않는 기준선 모델입니다.                | 5 장      |
| <code>np.zeros()</code>                 | 0 으로부터 채운 배열을 만듭니다. 무조건 정상 예측을 흉내 냅니다. | 5 장      |

### 교안에 없지만 코드에 나오는 옵션

실습 노트북과 이 책의 코드에는 교안 슬라이드에 적히지 않은 옵션이 몇 개 섞여 있습니다. 없어도 코드는 돌아가지만, 있으면 결과가 보기 좋아지거나 재현이 쉬워지는 것들입니다. **무엇을 하는 옵션인지만 짧게 적어 둡니다.**

| 옵션                                                       | 하는 일                                                                      |
|----------------------------------------------------------|---------------------------------------------------------------------------|
| <code>pd.set_option("display.max_colwidth", 1000)</code> | 표를 출력할 때 셀 내용이 중간에 잘리지 않도록 표시 폭을 늘립니다. 화면 출력에만 영향을 주고 데이터는 그대로입니다.        |
| <code>set_config(display="text")</code>                  | scikit-learn 모델을 출력할 때 그림 대신 글자로 보여 줍니다. 노트북 밖에서도 똑같이 읽히게 하는 설정입니다.       |
| <code>index=feats</code>                                 | <code>pd.Series</code> 를 만들 때 각 값에 이름을 붙입니다. 중요도 숫자 옆에 센서 이름이 함께 나오게 합니다. |

| 옵션                           | 하는 일                                              |
|------------------------------|---------------------------------------------------|
| <code>ascending=False</code> | 정렬 방향을 큰 값부터로 뒤집습니다. 기본값은 작은 값부터입니다.              |
| <code>normalize=True</code>  | <code>value_counts()</code> 의 결과를 개수 대신 비율로 바꿉니다. |

## 지난 회차에서 이어지는 것

| 도구                                             | 오늘 하는 일                                   | 다른 회차 |
|------------------------------------------------|-------------------------------------------|-------|
| <code>train_test_split(..., stratify=y)</code> | 어제 만든 네 덩어리를 오늘 그대로 모델에 넣습니다.             | 37 일차 |
| <code>(df["RUL"] &lt;= 30).astype(int)</code>  | <code>failure_soon</code> 라벨을 다시 만들어 씁니다. | 37 일차 |
| <code>value_counts(normalize=True)</code>      | 1,000 행 데이터의 불균형을 다시 확인합니다.               | 37 일차 |
| <code>sort_values()</code>                     | 확률 순으로 정렬해 정비 우선순위를 만듭니다.                 | 19 일차 |
| <code>pd.DataFrame({...})</code>               | 예측·실제·확률을 한 표로 묶습니다.                      | 17 일차 |

## CONTENTS

# 목차

오늘 배운 것의 지도

| 장    | 제목                               | 무엇을 배우나                                                                  |
|------|----------------------------------|--------------------------------------------------------------------------|
| 0 장  | 오늘의 지도                           | 오늘 쓰이는 용어를 먼저 정리하고, 모델을 만드는 흐름과 의심하는 흐름을 한 장의 그림으로 잡습니다.                 |
| 1 장  | 랜덤 포레스트의 구조                      | 결정 트리 하나의 약점에서 출발해 왜 숲을 쓰는지, 트리를 몇 그루 심을지 정합니다.                          |
| 2 장  | 학습과 예측                           | <code>fit</code> 으로 규칙을 채우고 <code>predict</code> 로 답을 받습니다. 재현성까지 확인합니다. |
| 3 장  | 확신도 — <code>predict_proba</code> | 같은 1 이라도 간신히 1 인지 확실한 1 인지 구분합니다. 0.5 부근이 왜 위험한지 봅니다.                    |
| 4 장  | 해석과 우선순위                         | 어떤 센서가 판단에 쓰였는지 확인하고, 예측·실제·확률을 한 표로 묶어 정비 순서를 만듭니다.                     |
| 5 장  | 정확도와 그 함정                        | 86.5%라는 점수가 좋은 점수인지 판단하는 법을 배웁니다. 기준선이 없으면 해석할 수 없습니다.                   |
| 부록 A | 치트시트                             | 오늘 나온 도구를 한 표로 압축했습니다.                                                   |
| 부록 B | 오류·함정 사전                         | 틀리기 쉬운 곳과, 실습 코드에서 잡아낸 조용한 오류 하나를 정리했습니다.                                |
| 부록 C | 실습 하이라이트 & 다음 시간 예고              | 오늘 실습에서 짚어 둘 것과 혼동행렬 예고입니다.                                              |

## CHAPTER 0 오늘의 지도

모델을 만들고, 그 점수를 의심하기까지

지난 시간까지 우리는 데이터를 준비했습니다. `failure_soon` 라벨을 만들고, 센서 6 개를 `X`로 정답을 `y`로 나누고, `stratify`로 비율을 지켜 학습용 2,891 행과 평가용 723 행으로 쪼개 두었습니다. **오늘은 그 준비물을 실제로 모델에 넣는 날입니다.**

그런데 오늘 수업은 한 가지를 더 합니다. 모델이 점수를 내놓은 뒤, 그 점수를 그대로 믿어도 되는지 묻습니다. 강사의 말대로 **"그동안 배웠던 것들의 한계점을 지적해 주는"** 시간이 마지막에 붙습니다. 그래서 오늘은 방향이 정반대인 두 흐름이 한 회차에 들어 있습니다.

### 0.1 오늘의 두 흐름

먼저 앞 네 장은 **모델을 만들고 그 결과를 현장 언어로 옮기는 흐름**입니다.



마지막 5 장은 **그 결과를 채점하고 의심하는 흐름**입니다. 방향이 반대입니다.



두 흐름이 만나는 지점이 오늘의 결론입니다. **잘 만든 모델이라도 점수 하나로는 잘 만들었다고 말할 수 없습니다.** 그 말을 하려면 비교 대상이 필요하고, 그것이 5 장의 베이스라인입니다.

### 0.2 용어 정리

오늘 처음 나오는 말이 많습니다. 본문에서 다시 자세히 다루지만, 여기서 한 번 훑고 가면 읽기가 훨씬 수월합니다.

| 용어           | 한 줄 뜻                             | 오늘 우리 데이터에서는              |
|--------------|-----------------------------------|---------------------------|
| 모델           | 입력을 받아 답을 내놓는 상자. 규칙은 학습으로 채워집니다. | 센서 6 개를 넣으면 0/1 이 나옵니다.   |
| 결정 트리        | 예/아니오 질문을 차례로 던져 답에 도달하는 방식.      | 센서값에 질문을 던져 가치를 나눕니다.     |
| 랜덤 포레스트      | 서로 다르게 자란 트리 여러 그루의 다수결.          | 트리 100 그루가 한 표씩 던집니다.     |
| n_estimators | 숲에 심을 트리 개수. 많을수록 안정적입니다.         | 100 — 입문 단계의 기본값          |
| 과적합          | 학습 데이터만 잘 맞히고 새 데이터에 약한 상태.       | 트리 한 그루를 쓰면 생기는 약점        |
| 확신도          | 같은 판정이라도 얼마나 확실한지 나타내는 확률.        | 100 그루 중 몇 그루가 1 에 투표했나   |
| 중요도          | 각 입력이 판단에 쓰인 정도. 합하면 1 이 됩니다.     | sensor_7 이 0.287 로 가장 큼   |
| 정확도          | 맞힌 개수 ÷ 전체 개수. 0 부터 1 사이입니다.      | 엔진 데이터에서 0.936            |
| 불균형          | 한 클래스가 다른 쪽보다 훨씬 많은 상태.           | 1,000 행 중 정상 781 · 고장 219 |
| 베이스라인        | 아무것도 학습하지 않은 기준선 모델.              | 무조건 정상이라고만 찍으면 78.0%      |
| 정확도의 함정      | 고장을 하나도 못 잡아도 점수가 높게 나오는 현상.      | 더미는 78 점인데 잡은 고장은 0 건     |

#### 개념

**오늘 마지막에 배운 것이 오늘 앞부분을 되돌아보게 만듭니다.** 1~4 장에서 만든 모델의 정확도는 93.6%였습니다. 좋은 숫자로 보입니다. 그런데 5 장을 배우고 나면 이런 질문이 남습니다 — 그 데이터에서 무조건 정상이라고만 찍었다면 몇 점이었을까요. 그 비교 없이는 93.6%가 잘한 것인지 알 수 없습니다.

## CHAPTER 1

# 랜덤 포레스트의 구조

왜 나무 한 그루가 아니라 숲인가

오늘 쓸 모델은 **랜덤 포레스트**입니다. 이름 그대로 숲인데, 숲을 이해하려면 나무 한 그루부터 봐야 합니다. 그 나무를 **결정 트리**라고 부릅니다.

## 1.1 결정 트리 — 스무고개로 판단하기

**개념** 결정 트리 (Decision Tree) 예/아니오 질문을 이어 답에 도달

**정의** 입력에 **예/아니오 질문을 차례로 던져** 답에 도달하는 방식입니다. 질문이 가지처럼 갈라져 나무 모양이 됩니다.

스무고개와 똑같습니다. 다리가 넷인가, 짙는가 하고 물어 가며 답을 줍니다. 우리 문제에서는 동물 대신 센서값에 질문을 던집니다.

```
# 트리가 던지는 질문의 모양 (개념)
sensor_4 > 1400 ?
├─ 예 → sensor_7 > 552 ?
│   ├── 예 → 1 (곧 고장)
│   └─ 아니오 → 0 (괜찮음)
└─ 아니오 → 0 (괜찮음)
```

중요한 것은 **이 질문의 기준을 사람이 정하지 않는다**는 점입니다. 어느 센서를 어떤 값에서 자를지, 트리가 데이터를 보고 스스로 고릅니다. 정답이 가장 깔끔하게 갈라지는 지점을 찾는 것이 기준입니다.

**왜 쓰나** 한 번 나눈 뒤에도 0 과 1 이 섞여 있으면 다른 센서로 또 나눕니다. 이것을 반복하면 **가지 끝에 같은 정답만 모인 묶음**이 남습니다. 그 상태가 학습이 끝난 트리입니다.

### 개념

37 일차에서 cycle 을 입력에서 뺀 이유가 여기서 다시 보입니다. 트리는 **기준보다 큰가 작은가만** 묻기 때문에, 설비마다 의미가 다른 값이 들어가면 엉뚱한 기준을 학습합니다.

**개념** 트리 하나의 약점 과적합 · 불안정

**정의** 결정 트리 한 그루는 **학습 데이터에 지나치게 맞춰지고**, 데이터가 조금만 바뀌어도 결과가 크게 흔들립니다.

문제집을 통째로 외운 학생을 떠올리면 됩니다. 본 문제는 완벽히 풀지만 조금만 비틀면 당황합니다. 37 일차 6 장에서 본 **과적합**이 바로 이것입니다.

**왜 쓰나** 그래서 트리 하나로는 현장에 내보내기 어렵습니다. 오늘 데이터로 확인해 보면 트리 한 그루의 정확도는 **89.2%**인데, 100 그루를 모으면 **93.6%**로 올라갑니다. 4.4%p 차이가 전부 이 약점에서 나옵니다.

**응용** 해결책은 단순합니다. **한 그루에 의존하지 말고 여러 그루의 의견을 모읍니다**. 각 트리가 데이터와 센서의 일부만 보고 자라게 하면 서로 다른 관점을 가지게 되고, 한 트리의 치우침이 전체 속에서 없어집니다.

## 1.2 숲 — 여러 트리의 다수결

**개념** 랜덤 포레스트 (Random Forest) 트리 여러 그루의 다수결

**정의** 서로 조금씩 다르게 자란 트리 **수십~수백 그루가 각각 0/1 로 투표**하고, 가장 많은 표를 받은 답이 최종 결론이 됩니다.

포레스트는 숲이라는 뜻이고, 랜덤은 각 트리가 데이터와 센서의 일부만 무작위로 보고 자란다는 뜻입니다. 그래서 트리마다 의견이 갈라지고, 그 덕분에 다수결이 의미를 가집니다.

랜덤 포레스트 — 각 트리가 한 표씩 던지고 많은 쪽이 결론 (개념 도식)



▲ 트리 다섯 그루의 투표 예시 — 한 그루가 틀려도 나머지가 제대로 판단하면 다수결 속에 묻힙니다 (개념 도식)

**왜 쓰나** 여럿이 의견을 모으면 한 트리의 실수가 나머지에 의해 바로잡힙니다. 데이터가 살짝 바뀌어도 결과가 확 흔들리지 않습니다. **집단지성이 오차를 보정하는 구조**입니다.

#### 개념

단위를 맞출 필요가 없는 이유는 트리의 질문 방식에 있습니다. 값의 크기를 서로 비교하는 것이 아니라 **기준값보다 큰지 작은지**만 묻기 때문에, sensor\_3 이 1,588 이고 sensor\_15 가 8.47 이어도 아무 문제가 없습니다.

#### 설비 적용

입문 모델로 랜덤 포레스트를 먼저 배우는 데는 이유가 있습니다. 설정할 것이 거의 없고, **센서마다 단위가 제각각이어도 그대로 작동하며**, 결과가 안정적이고, 어떤 센서가 중요했는지까지 알려 줍니다. 온도·압력·진동이 뒤섞인 설비 데이터에 특히 잘 맞습니다.

## 1.3 트리를 몇 그루 심을 것인가

**문법** `n_estimators` 숲에 심을 트리 개수

**정의** estimator는 트리 하나, n은 개수입니다. **숲에 트리를 몇 그루 심을지** 정하는 설정입니다.

**형태** `RandomForestClassifier(n_estimators=100, random_state=42)`

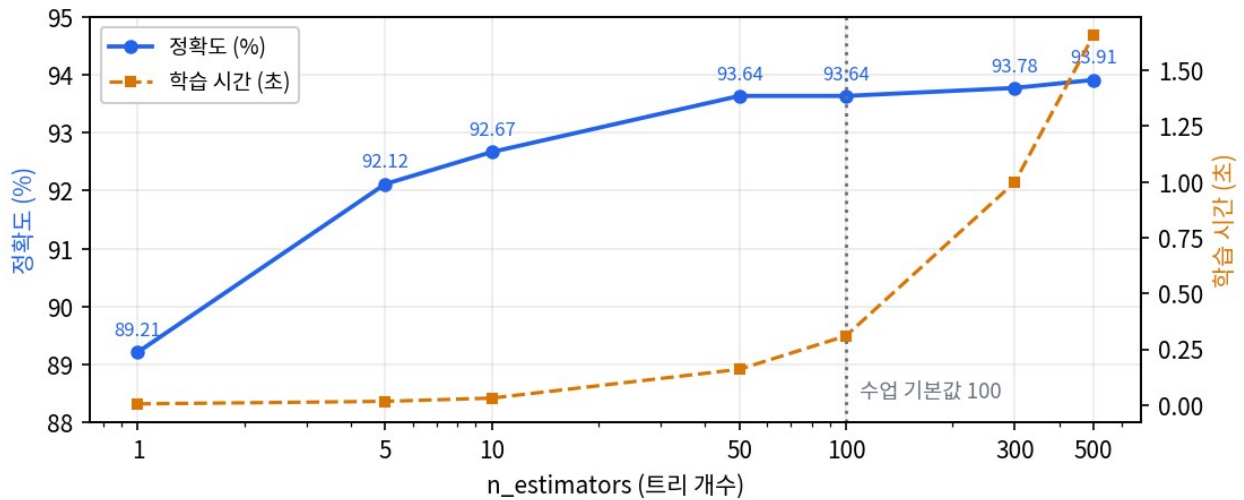
많을수록 다수결이 안정적입니다. 그런데 무한정 늘리면 어떻게 될까요. 값을 바꿔 가며 직접 재 봤습니다.

```
import time
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

for n in [1, 5, 10, 50, 100, 300, 500]:
    t0 = time.time()
    m = RandomForestClassifier(n_estimators=n, random_state=42).fit(X_train, y_train)
    sec = time.time() - t0
    # time.time()은 현재 시각(초) - 두 번 재서 빼면 걸린
    시간
    print(f"n={n:4d}   정확도={accuracy_score(y_test, m.predict(X_test)):.4f}   학습={sec:.3f}
    초")
```

```
▶ n=   1   정확도=0.8921   학습=0.005 초
▶ n=   5   정확도=0.9212   학습=0.016 초
▶ n=  10   정확도=0.9267   학습=0.031 초
▶ n=  50   정확도=0.9364   학습=0.160 초
▶ n= 100   정확도=0.9364   학습=0.310 초
▶ n= 300   정확도=0.9378   학습=0.999 초
▶ n= 500   정확도=0.9391   학습=1.656 초
```

1 그루에서 50 그루까지는 정확도가 **89.2%에서 93.6%로 4.4%p** 뛰어오릅니다. 그런데 50 에서 500 까지 열 배를 늘려도 **93.6%에서 93.9%로 0.3%p** 밖에 안 오릅니다. 학습 시간은 그동안 0.16 초에서 1.66 초로 **열 배**가 됐습니다.



▲ 파란 선이 정확도, 주황 점선이 학습 시간 — 50 그루쯤에서 정확도가 평평해지는데 시간만 계속 올라갑니다

**왜 쓰나** 그래서 100 이라는 기본값이 나옵니다. **다수결의 안정성은 충분히 누리면서 학습이 금세 끝나는 지점**입니다. 이번 수업에서 개수 고민 없이 100 을 쓰는 이유가 이 그림에 있습니다.

#### 주의

이 그래프의 모양은 데이터마다 다릅니다. 데이터가 크고 복잡하면 평평해지는 지점이 더 오른쪽으로 갑니다. **기본값 100 으로 시작하되, 프로젝트에서는 이렇게 한 번 재 보고 정하는 것이** 근거 있는 선택입니다.

## 1.4 트리 한 그루를 열어 보기

숲은 트리 100 그루라 통째로 들여다볼 수 없습니다. 그런데 **트리 한 그루만 알게 학습시켜 보면** 안에서 어떤 질문이 만들어지는지 직접 볼 수 있습니다. `max_depth=2`는 질문을 두 단계까지만 하라는 뜻이고, `export_text`는 학습된 트리를 글자로 그려 주는 함수입니다.

```
from sklearn.tree import DecisionTreeClassifier, export_text

tree = DecisionTreeClassifier(max_depth=2, random_state=42).fit(X_train, y_train)
print(export_text(tree, feature_names=feats))
```

```

> |--- sensor_7 <= 552.40
> |   |--- sensor_11 <= 47.84
> |   |   |--- class: 0
> |   |   |--- sensor_11 > 47.84
> |   |   |--- class: 1
> |--- sensor_7 > 552.40
> |   |--- sensor_15 <= 8.54
> |   |   |--- class: 0
> |   |   |--- sensor_15 > 8.54
> |   |   |--- class: 0
```

첫 질문이 `sensor_7` 이 552.40 보다 작은가입니다. 아무도 552.40 이라는 숫자를 알려 주지 않았습니다. 트리가 3 천 개 가까운 행을 훑어 정답이 가장 깔끔하게 갈라지는 지점을 직접 찾은 값입니다. 그다음 질문은 왼쪽 가지에서는 `sensor_11`, 오른쪽 가지에서는 `sensor_15` 로 서로 다릅니다.



가지 끝을 보면 1 이 나오는 길은  $\text{sensor\_7} \leq 552.40$  이면서  $\text{sensor\_11} > 47.84$  하나뿐입니다. 이 얇은 트리 한 그루만으로도 정확도가 90.2%나 나옵니다. 숲(93.6%)에는 못 미치지만, 질문 두 번으로 열에 아홉을 맞히는 셈입니다.

#### 개념

4장에서 보게 될 **중요도**가 여기서 미리 설명됩니다. 이 트리가 가장 먼저 고른 센서가  $\text{sensor\_7}$  이고, 실제로 100 그루 숲에서도  $\text{sensor\_7}$  의 중요도가 0.287 로 1 위입니다. 자주, 그리고 위쪽에서 쓰인 센서가 중요도가 높게 나옵니다.

#### 주의

`max_depth`와 `export_text`는 교안에 나오지 않는 도구입니다. `max_depth`는 질문을 몇 단계까지 던질지 제한하는 옵션이고(제한하지 않으면 가지가 너무 깊어져 글자로 못 씁니다), `export_text`는 학습된 트리를 글자 그림으로 출력하는 함수입니다. 숲 안을 들여다보려고 잠깐 빌려 쓴 것이지 오늘 외울 도구는 아닙니다.

## 1 장 한 장 정리

| 배운 것                      | 한 줄 정의                                            | 기억할 숫자            |
|---------------------------|---------------------------------------------------|-------------------|
| 결정 트리                     | 예/아니오 질문을 이어 붙여 답에 도달하는 나무 구조                     | 한 그루 정확도 0.8921   |
| 트리의 약점                    | 데이터가 조금만 바뀌어도 가지가 통째로 달라짐                         | —                 |
| 랜덤 포레스트                   | 조금씩 다르게 자란 트리들의 다수결                               | 100 그루 정확도 0.9364 |
| <code>n_estimators</code> | 숲에 심을 트리 개수                                       | 50 에서 평탄 · 기본 100 |
| 트리 속 질문                   | 학습이 스스로 찾은 경계값 ( $\text{sensor\_7} \leq 552.40$ ) | 깊이 2 만으로 0.9018   |

#### 한 장 정리

1 장의 결론은 **한 그루로는 부족하고 여럿을 모으면 안정된다**는 것 하나입니다. 4.4%p의 차이가 그 값이고, 50 그루를 넘기면 더 모아도 거의 오르지 않습니다. 다음 장에서는 이 숲에 실제로 데이터를 넣어 학습시킵니다.

## CHAPTER 2

# 학습과 예측

빈 상자에 규칙을 채우고 답을 받기

모델을 만들었다고 해서 무언가를 아는 것은 아닙니다. 갓 만든 모델은 **받은 것 같지만 씨앗은 안 뿌린 상태**입니다. 안에 든 트리 100 그루는 아직 백지이고, 어떤 센서가 중요한지도 어디서 자를지도 모릅니다. 진짜 학습은 `fit` 에서 시작합니다.

## 2.1 fit — 규칙을 채우는 한 줄

**메서드** `model.fit()` 학습용 데이터로 규칙 채우기

**정의** 학습용 입력과 정답을 건네면 **트리 100 그루가 동시에 자라며** 각자 판단 규칙을 만듭니다.

**형태** `model.fit(X_train, y_train)`

어제 만들어 둔 네 덩어리를 그대로 씁니다. 데이터 준비를 다시 하고 모델을 만든 뒤 학습시키는 데까지가 아래 코드입니다.

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

df = pd.read_csv("../Data/27_cmapss_fd001_sample.csv", encoding="utf-8")
df["failure_soon"] = (df["RUL"] <= 30).astype(int)

feats = ["sensor_2", "sensor_3", "sensor_4", "sensor_7", "sensor_11", "sensor_15"]
X = df[feats]
y = df["failure_soon"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

▶ RandomForestClassifier(random_state=42)
```

출력에 `n_estimators=100` 이 안 보이는 것은 그것이 **기본값이라 생략된 것**입니다. 기본값과 다른 설정만 표시하는 것이 scikit-learn 의 출력 방식입니다. 학습은 샘플 데이터 기준 0.3 초면 끝납니다.

**왜 쓰나** 이 한 줄이 **어떤 센서값일 때 정답이 1 이고 0 인지를 익히는 단계**입니다. 학습이 끝난 모델 안에는 데이터에서 찾아낸 판단 규칙, 즉 잘 자란 트리 100 그루가 들어 있습니다.

### 개념

모델 속 규칙은 **우리가 보여 준 데이터에서 나온 것**입니다. 라벨이 잘못됐거나 값이 빠져 있으면 그대로 모델 안에 들어갑니다. 37 일차에 라벨 만들기과 누수 제거에 공을 들인 이유가 여기 있습니다.

### 자주 하는 실수

`fit` 에는 **학습용만 넣습니다**. 평가용을 넣으면 시험 문제를 미리 보여 준 셈이라 평가가 의미를 잃습니다. 그리고 같은 데이터로 `fit` 을 여러 번 불러도 더 똑똑해지지 않습니다 — 처음부터 다시 학습될 뿐입니다.

## 2.2 predict — 새 데이터에 답하기

**메서드** `model.predict()` 0 또는 1 판단 받기

**정의** 입력의 각 행에 대해 **0 또는 1** 판단을 배열로 돌려줍니다.

**형태** `y_pred = model.predict(X_test)`

학습에 쓰지 않고 아껴 둔 평가용 723 행을 넣어 봅니다.

```
y_pred = model.predict(X_test)

print("평가용 행 수:", len(y_test))
print("예측 개수 :", len(y_pred))
print("예측 앞 10 :", y_pred[:10])
print("실제 앞 10 :", y_test.head(10).values)
```

```
▶ 평가용 행 수: 723
▶ 예측 개수 : 723
▶ 예측 앞 10 : [0 0 0 0 0 0 0 0 1 1]
▶ 실제 앞 10 : [0 0 0 0 0 0 0 0 1 1]
```

**예측 개수와 평가용 행 수가 같습니다.** 예측은 입력과 같은 순서로 나오므로, 첫 번째 예측은 첫 번째 행의 답입니다. 앞 10 개만 봐도 열 자리가 모두 일치합니다.

**왜 쓰나** fit 은 오래 걸리는 공부 단계이고, predict 는 이미 만든 규칙에 대입하는 단계라 거의 즉시 끝납니다. 한 번 학습해 둔 모델은 새 입력이 들어올 때마다 빠르게 답을 냅니다.

#### 주의

predict 에 넣는 입력은 **학습 때와 같은 형태**여야 합니다. 센서 6 개로 학습했다면 예측도 같은 6 개를 같은 순서로 넣어야 합니다. 컬럼 하나를 빼거나 순서를 바꾸면 오류가 나거나, 더 나쁘게는 조용히 엉뚱한 답이 나옵니다.

## 2.3 random\_state — 같은 결과가 다시 나오는가

랜덤 포레스트에는 무작위가 들어 있습니다. 각 트리가 데이터와 센서의 **일부만 무작위로 골라** 자라기 때문입니다. 그러면 돌릴 때마다 결과가 달라질까요. 직접 확인해 봅니다.

```
model2 = RandomForestClassifier(n_estimators=100, random_state=7).fit(X_train, y_train)
model3 = RandomForestClassifier(n_estimators=100, random_state=42).fit(X_train, y_train)
```

```
y_pred2 = model2.predict(X_test)
y_pred3 = model3.predict(X_test)
```

```
print("42 vs 7 앞 10 :", y_pred[:10], y_pred2[:10])
print("42 vs 7 전체 불일치:", (y_pred != y_pred2).sum(), "/", len(y_pred))
print("42 재현 완전 동일:", (y_pred == y_pred3).all())
```

```
▶ 42 vs 7 앞 10 : [0 0 0 0 0 0 0 0 1 1] [0 0 0 0 0 0 0 0 1 1]
▶ 42 vs 7 전체 불일치: 14 / 723
▶ 42 재현 완전 동일: True
```

세 가지가 한 번에 확인됩니다. 첫째, random\_state 가 같으면 결과가 완전히 똑같습니다. 둘째, 값을 7 로 바꾸면 723 건 중 14 건의 판정이 달라집니다. 셋째, 그 14 건은 앞 10 개에는 없습니다 — 확신이 강한 행은 어느 숲에서나 같은 답이 나오고, 흔들리는 것은 경계에 걸친 행들입니다.

#### 연결

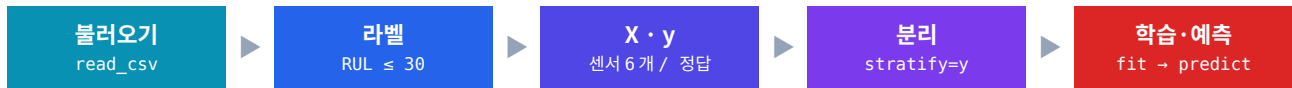
이 14 건이 3 장에서 다룰 **확신도**와 바로 이어집니다. 100 그루 중 51 그루만 1 에 투표한 행이라면, 숲을 다시 심었을 때 49 대 51 로 뒤집히기 쉽습니다. 반대로 98 그루가 1 에 투표한 행은 어지간해서는 바뀌지 않습니다. 판정이 흔들린다는 것 자체가 그 행이 애매하다는 신호입니다.

### 주의

`random_state`는 성능을 올리는 옵션이 아닙니다. 42가 7보다 좋은 값이 아니라, 같은 값을 쓰면 같은 결과가 나온다는 약속일 뿐입니다. 리포트에 적은 숫자를 나중에 누가 다시 돌려도 똑같이 나와야 하므로, 팀 안에서 하나로 통일해 두고 바꾸지 않는 것이 맞습니다.

## 2.4 여섯 단계를 한 흐름으로

37일차와 오늘을 이으면 불러오기부터 예측까지 여섯 단계가 완성됩니다. 흩어져 배운 것을 한 덩어리로 붙여 보면 전체 그림이 잡힙니다.



```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

# 1. 불러오기
df = pd.read_csv("../Data/27_cmapss_fd001_sample.csv", encoding="utf-8")

# 2. 라벨 — 37일차에서 정한 임계값 30
df["failure_soon"] = (df["RUL"] <= 30).astype(int)

# 3. X / y — RUL·unit_id·cycle 은 넣지 않는다 (정답 누수)
feats = ["sensor_2", "sensor_3", "sensor_4", "sensor_7", "sensor_11", "sensor_15"]
X, y = df[feats], df["failure_soon"]

# 4. 분리 — 비율을 지켜 나눈다
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 5. 학습
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# 6. 예측
y_pred = model.predict(X_test)

print("학습용", X_train.shape, "· 평가용", X_test.shape)
print("1로 예측된 행:", int((y_pred == 1).sum()), "건")

▶ 학습용 (2891, 6) · 평가용 (723, 6)
▶ 1로 예측된 행: 114 건
```

스무 줄 남짓입니다. 그중 모델과 직접 관련된 줄은 세 줄뿐이고 나머지는 전부 데이터 준비입니다. 실무에서 시간을 가장 많이 쓰는 곳이 앞단이라는 말이 이 비율에서 그대로 보입니다.

| 단계       | 한 일                 | 건너뛰면                     |
|----------|---------------------|--------------------------|
| 1. 불러오기  | CSV를 표로 읽습니다.       | —                        |
| 2. 라벨    | 연속된 RUL을 0/1로 접습니다. | 맞힐 정답이 없습니다.             |
| 3. X / y | 입력과 정답을 갈라 담습니다.    | RUL이 X에 남으면 정답 누수가 됩니다.  |
| 4. 분리    | 학습용과 평가용을 나눕니다.     | 배운 문제로 시험 봐서 점수가 부풀려집니다. |

| 단계    | 한 일                  | 건너뛰면 |
|-------|----------------------|------|
| 5. 학습 | 트리 100 그루에 규칙을 채웁니다. | —    |
| 6. 예측 | 평가용에 답을 받습니다.        | —    |

### 연결

2~4 단계가 37 일차 내용 전부입니다. 그때는 모델 없이 준비만 해서 왜 이렇게까지 하나 싶었지만, 오늘 5~6 단계를 붙여 보면 **앞단이 잘못되면 뒷단이 아무리 잘 돌아가도 소용없다**는 것이 분명한겁니다.

## 2 장 한 장 정리

| 배운 것                                  | 한 줄 정의                                 | 기억할 숫자       |
|---------------------------------------|----------------------------------------|--------------|
| <code>RandomForestClassifier()</code> | 설정만 담긴 빈 모델 상자를 만든다                    | 트리 100 그루    |
| <code>.fit(X_train, y_train)</code>   | 문제와 정답을 함께 주어 규칙을 채운다                  | 학습용 2,891 행  |
| <code>.predict(X_test)</code>         | 새 입력마다 0 또는 1 판정을 배열로 받는다              | 평가용 723 행    |
| <code>random_state</code>             | 무작위를 고정해 같은 결과가 다시 나오게 한다              | 42↔7 차이 14 건 |
| 전체 흐름                                 | 불러오기 → 라벨 → $X \cdot y$ → 분리 → 학습 → 예측 | 여섯 단계 · 20 줄 |

### 한 장 정리

2 장의 결론은 **모델을 만드는 코드는 세 줄뿐**이라는 것입니다. 나머지는 전부 데이터 준비입니다. 그리고 판정이 흔들린 14 건은 우연이 아니라 **원래 애매했던 행**인데, 그 애매함을 숫자로 꺼내 보는 것이 다음 장입니다.

predict 가 돌려준 것은 0 과 1 뿐입니다. 그런데 같은 1 이라도 **100 그루 중 98 그루가 1 에 투표한 1** 과 **51 그루만 1 에 투표한 1** 은 성격이 전혀 다릅니다. 앞의 것은 확실하고 뒤의 것은 아슬아슬합니다. 그 차이를 숫자로 꺼내 주는 것이 predict\_proba 입니다.

### 3.1 predict\_proba — 투표 비율 꺼내기

**메서드** model.predict\_proba() 0 일 확률과 1 일 확률

**정의** 각 행에 대해 [0 일 확률, 1 일 확률] 두 값을 돌려줍니다. 100 그루 중 몇 그루가 그쪽에 투표했는지의 비율입니다.

**형태** proba = model.predict\_proba(X\_test)

두 값을 더하면 항상 1 이 됩니다. 100 그루가 둘 중 하나에 반드시 투표하기 때문입니다.

```
proba = model.predict_proba(X_test)
print(proba[:5].round(2))
```

```
▶ [[1.    0.   ]
   [1.    0.   ]
   [0.86  0.14]
   [1.    0.   ]
   [1.    0.   ]]
```

세 번째 행만 조금 다릅니다. **100 그루 중 14 그루가 곧 고장에 투표했습니다**. 나머지 네 행은 100 그루가 만장일치로 정상이라고 했습니다.

**왜 쓰나** 0/1 판정만으로는 이 차이가 보이지 않습니다. 세 번째 행도 최종 판정은 0 이지만, **완전히 안심할 행은 아니라는** 정보가 확률에 남아 있습니다.

**문법** proba[:, 1] 1 일 확률만 꺼내기

**정의** 2 차원 배열에서 **모든 행의 두 번째 열만** 골라냅니다. 앞의 : 가 전체 행, 뒤의 1 이 두 번째 열입니다.

**형태** proba[:, 1]

우리가 관심 있는 것은 곧 고장일 확률, 즉 두 번째 열입니다. 0 일 확률은 1 에서 빼면 나오므로 따로 볼 필요가 없습니다.

```
proba_1 = proba[:, 1]
print(proba_1[:10].round(2))
```

```
▶ [0.    0.    0.14 0.    0.    0.02 0.    0.21 0.85 0.56]
```

아홉 번째와 열 번째가 1 로 판정된 행입니다. 그런데 **0.85 와 0.56** 으로 확신의 세기가 꽤 다릅니다. 앞의 것은 85 그루가 동의했고 뒤의 것은 56 그루만 동의했습니다.

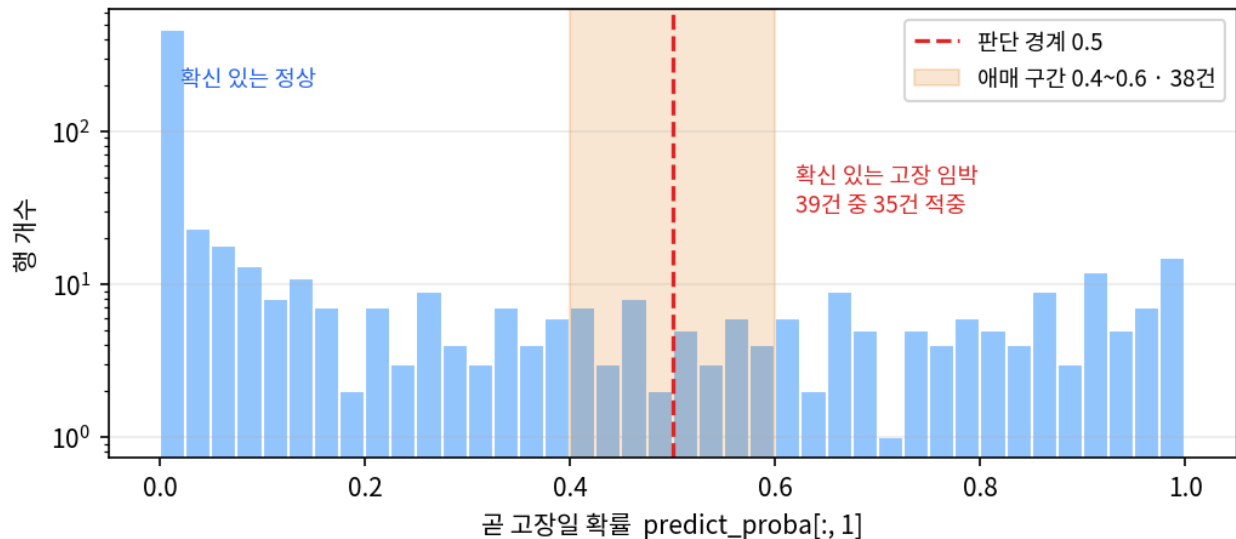
**왜 쓰나** 정비 현장에서는 이 차이가 곧 순서가 됩니다. 확률 0.85 인 설비를 먼저 보고, 0.56 인 설비는 다음 주기에 다시 확인하는 식입니다. **단순한 0/1 보다 훨씬 풍부한 판단 재료**입니다.

#### 주의

대괄호 안의 쉼표를 빠뜨려 proba[:1]로 쓰면 **첫 번째 행 하나만** 나옵니다. 쉼표 하나 차이로 전혀 다른 결과가 나오는데 오류는 나지 않으니 주의하십시오.

### 3.2 확률은 어떻게 퍼져 있나

723 개 행의 확률이 어디에 몰려 있는지 보면, 모델이 무엇을 확신하고 무엇을 망설이는지가 한눈에 드러납니다.



▲ 세로축은 로그 눈금입니다 — 양 끝의 확신 구간에 대부분이 몰려 있고 가운데는 성깁니다. 주황 띠가 0.4~0.6 애매 구간으로 38 건입니다

양쪽 끝이 압도적으로 높습니다. 대부분의 행은 모델이 확신을 가지고 판단합니다. 문제는 가운데입니다. 확률 0.4 에서 0.6 사이에 38 건이 걸려 있는데, 이 38 건 중 실제로 고장 임박이었던 것은 22 건입니다. 절반을 조금 넘습니다 — 동전 던지기보다 아주 조금 나은 수준입니다.

```
import pandas as pd

res = pd.DataFrame({"예측": y_pred, "실제": y_test.values, "곧고장확률": proba_1})

amb = res[(res["곧고장확률"] >= 0.4) & (res["곧고장확률"] <= 0.6)]
print("애매 구간 0.4~0.6 :", len(amb), "건 · 그중 실제 고장", int(amb["실제"].sum()), "건")

sure = res[res["곧고장확률"] >= 0.9]
print("확신 구간 0.9 이상 :", len(sure), "건 · 그중 실제 고장", int(sure["실제"].sum()), "건")
```

- ▶ 애매 구간 0.4~0.6 : 38 건 · 그중 실제 고장 22 건
- ▶ 확신 구간 0.9 이상 : 39 건 · 그중 실제 고장 35 건

두 구간을 나란히 놓으면 확신도가 왜 쏠모 있는지 분명해집니다. 확률 0.9 이상인 39 건은 35 건이 진짜 고장이었습니다. 열 번 찍으면 아홉 번은 맞습니다. 반면 애매 구간 38 건은 22 건, 즉 열 번에 여섯 번 정도입니다. 같은 모델이 낸 예측인데 신뢰도가 이렇게 다릅니다.

| 확률 구간     | 건수   | 실제 고장        | 읽는 법                  | 현장 조치                |
|-----------|------|--------------|-----------------------|----------------------|
| 0.9 이상    | 39 건 | 35 건 (89.7%) | 100 그루 중 90 그루 이상이 동의 | 즉시 점검 대상             |
| 0.6 ~ 0.9 | —    | —            | 다수가 동의하나 이견 있음        | 우선순위 목록에 포함          |
| 0.4 ~ 0.6 | 38 건 | 22 건 (57.9%) | 투표가 거의 반반으로 갈림        | 한 번 더 지켜봄 · 다음 주기 관찰 |
| 0.4 미만    | —    | —            | 대다수가 정상에 투표           | 정상 운전 계속             |

#### 설비 적용

이 표가 곧 운영 규칙이 됩니다. 확률 0.9 이상은 바로 정비 지시를 내고, 0.4~0.6은 지시 대신 관찰 목록에 올려 다음 사이클의 데이터를 한 번 더 봅니다. 모델이 애매하다고 말하는 행에 확정적인 조치를 걸면 헛걸음이 늘어납니다. 모델이 자기 확신의 정도를 알려 준다는 사실 자체를 운영에 쓰는 것입니다.

#### 연결

2장에서 random\_state 를 7 로 바꿨을 때 판정이 달라진 14 건을 기억하십시오. 그 행들이 대부분 이 애매 구간에 있습니다. 숲을 다시 심으면 뒤집히는 행과 확률이 0.5 근처인 행은 같은 행입니다. 확신도는 결국 그 흔들림을 숫자로 옮긴 것입니다.

### 3.3 판단 경계를 0.5 에서 옮겨 보기

predict 는 확률이 0.5 를 넘으면 1 로 판정합니다. 그런데 0.5 라는 숫자에 특별한 근거가 있는 것은 아닙니다. 확률을 직접 가지고 있으니 경계를 우리가 정할 수 있습니다.

```
for 경계 in [0.3, 0.5, 0.7]:
    판정 = (proba_1 >= 경계).astype(int)      # 확률이 경계 이상이면 1
    맞힘 = ((판정 == 1) & (y_test.values == 1)).sum()  # 진짜 고장을 고장이라 함
    헛걸음 = ((판정 == 1) & (y_test.values == 0)).sum()  # 정상인데 고장이라 함
    놓침 = ((판정 == 0) & (y_test.values == 1)).sum()  # 고장인데 정상이라 함
    print(f"경계 {경계} 1 판정 {판정.sum():3d}건 맞힘 {맞힘:3d} 헛걸음 {헛걸음:2d} 놓침 {놓침:2d}")
```

- ▶ 경계 0.3 1 판정 156 건 맞힘 116 헛걸음 40 놓침 8
- ▶ 경계 0.5 1 판정 116 건 맞힘 98 헛걸음 18 놓침 26
- ▶ 경계 0.7 1 판정 77 건 맞힘 70 헛걸음 7 놓침 54

평가용 723 행 중 **실제 고장 임박은 124 건**입니다. 경계를 낮추면 그물을 넓게 치는 것이고, 높이면 확실한 것만 건지는 것입니다. 어느 쪽도 공짜가 아닙니다.

| 경계  | 1로 판정 | 맞힘  | 헛걸음 | 놓침 | 정확도    | 성격     |
|-----|-------|-----|-----|----|--------|--------|
| 0.3 | 156 건 | 116 | 40  | 8  | 0.9336 | 넓게 친다  |
| 0.5 | 116 건 | 98  | 18  | 26 | 0.9391 | 기본값    |
| 0.7 | 77 건  | 70  | 7   | 54 | 0.9156 | 확실한 것만 |

경계 0.3 은 124 건 중 **116 건을 건져 놓친 것이 8 건**뿐입니다. 대신 정상 설비 40 대를 헛되이 뜯어보게 됩니다. 경계 0.7 은 헛걸음이 7 건으로 줄지만 **54 건을 놓칩니다** — 고장 임박의 절반 가까이를 못 본 채 지나갑니다.

#### 설비 적용

**어느 쪽 실수가 더 비싼가로 결정합니다.** 점검 한 번이 30 분이고 고장 한 번이 라인 정지 여덟 시간이라면 놓침이 훨씬 비싸므로 경계를 0.3 쪽으로 내립니다. 반대로 점검에 설비를 멈추고 분해해야 한다면 헛걸음이 비싸므로 경계를 올립니다. **모델을 다시 학습시킬 필요 없이 숫자 하나만 바꾸면 되는 조정**이라는 점이 중요합니다.

#### 연결

표의 맞힘·헛걸음·놓침 세 칸이 바로 다음 시간에 배울 **혼동행렬**입니다. 정확도 하나로는 이 셋을 구분할 수 없습니다. 실제로 경계 0.3(0.9336)과 0.7(0.9156)은 정확도가 비슷한데 **속은 완전히 다릅니다**. 정확도만 보면 이 차이가 통째로 사라집니다 — 5장에서 그 이야기를 합니다.

#### 주의

표의 0.5 줄(0.9391)이 predict 의 정확도(0.9364)와 미세하게 다릅니다. 확률이 **정확히 0.50 인 행이 2 건** 있는데, proba\_1 >= 0.5 는 이 둘을 1 로 넣고 predict 는 0 으로 처리하기 때문입니다. 둘 다 틀린 것이 아니라 동점 처리 규칙이 다를 뿐입니다.

## 3 장 한 장 정리

| 배운 것              | 한 줄 정의                         | 기억할 숫자         |
|-------------------|--------------------------------|----------------|
| .predict_proba(X) | 행마다 [0 일 확률, 1 일 확률] 두 칸을 돌려준다 | 723 × 2 모양     |
| proba[:, 1]       | 모든 행에서 1 일 확률 열만 뽑는다           | 심표 빠뜨리면 1 행만   |
| 확신 구간             | 100 그루 중 90 그루 이상이 동의한 행       | 39 건 중 35 건 적중 |
| 애매 구간             | 투표가 거의 반반으로 갈린 행               | 38 건 중 22 건 적중 |



| 배운 것  | 한 줄 정의                  | 기억할 숫자          |
|-------|-------------------------|-----------------|
| 판단 경계 | 1 로 볼 확률의 기준선 — 우리가 정한다 | 0.3 / 0.5 / 0.7 |

#### 한 장 정리

3 장의 결론은 **같은 1 이라도 값이 다르다**는 것입니다. 확률을 가지고 있으면 순서를 매길 수 있고 경계를 옮길 수도 있습니다. 경계를 0.3 으로 내리면 놓침이 26 건에서 8 건으로 줄지만 헛걸음이 18 건에서 40 건으로 늘어납니다 — **공짜인 선택은 없습니다.**

모델이 답을 냈습니다. 이제 남은 일은 그 답을 **정비팀이 바로 쓸 수 있는 형태로** 옮기는 것입니다. 두 가지가 필요합니다 — **어디를 봐야 하는지와 무엇부터 봐야 하는지**입니다.

## 4.1 어떤 센서가 판단에 쓰였나

**속성** `model.feature_importances_` 각 입력이 판단에 쓰인 정도

**정의** 학습이 끝난 모델이 가지고 있는 속성입니다. 각 입력이 정답을 가르는 데 얼마나 쓰였는지 점수로 알려 줍니다. 괄호를 붙이지 않습니다.

**형태** `model.feature_importances_`

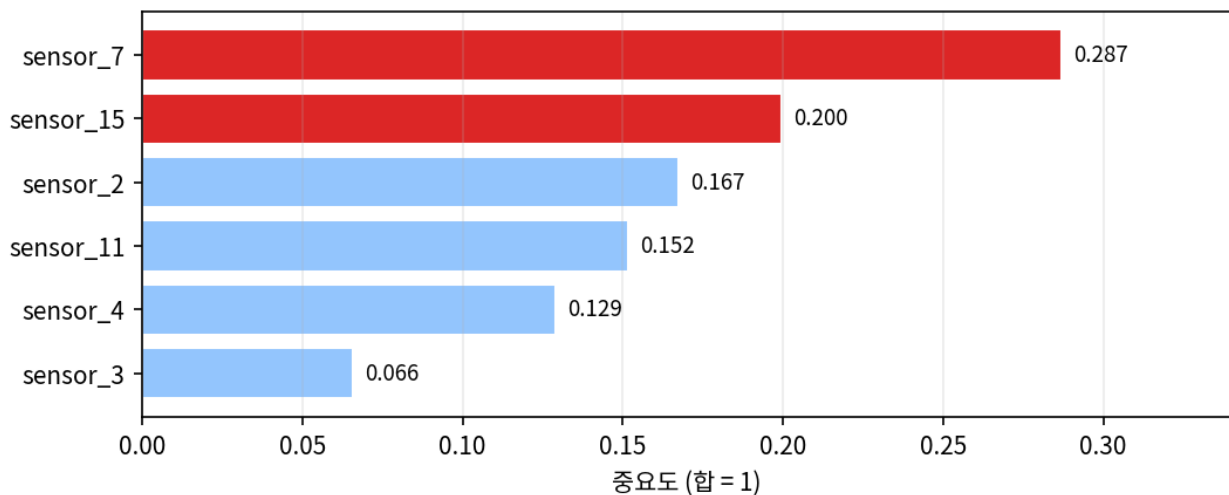
점수의 합은 1 이므로 **비중처럼 읽을 수 있습니다**. 숫자만 나오면 어느 센서인지 알 수 없으니 이름을 붙여 정렬합니다.

```
import pandas as pd

importance = pd.Series(model.feature_importances_, index=feats)
print(importance.sort_values(ascending=False))
```

```
▶ sensor_7      0.286797
▶ sensor_15     0.199502
▶ sensor_2      0.167326
▶ sensor_11     0.151536
▶ sensor_4      0.129044
▶ sensor_3      0.065796
▶ dtype: float64
```

`sensor_7` 과 `sensor_15` 둘이 합쳐 **0.486**, 판단의 절반 가까이를 차지합니다. 반대로 `sensor_3` 은 0.066 으로 거의 쓰이지 않았습니다. 여섯 개를 똑같이 넣었는데 모델이 알아서 무게를 다르게 둔 것입니다.



▲ 위 두 센서(빨강)가 판단의 절반 가까이를 차지합니다 — 아래 `sensor_3` 은 거의 쓰이지 않았습니다

**왜 쓰나** 랜덤 포레스트가 입문 모델로 좋은 이유 중 하나가 이것입니다. 많은 모델은 왜 그렇게 판단했는지 알려 주지 않는 블랙박스인데, 랜덤 포레스트는 **판단 근거를 사람이 점검할 수 있게** 열어 줍니다.

### 설비 적용

그래도 현장에는 충분히 쓸모가 있습니다. 정비팀에 **1 로 예측된 설비 목록만** 넘기는 것과, **그 설비의 sensor\_7 · sensor\_15 부위를 먼저 보라**는 말을 함께 넘기는 것은 다릅니다. 어디를 왜 살펴야 하는지까지 전달될 때 분석이 현장 행동이 됩니다.

### 자주 하는 실수

중요도가 높다는 것은 그 센서가 고장의 원인이라는 뜻이 아닙니다. 판단에 많이 쓰였다는 뜻일 뿐입니다. 고장 직전에 함께 변하는 좋은 단서이지만, 진짜 원인인지는 설비 전문가가 확인해야 합니다. 중요도만 보고 원인을 단정하면 엉뚱한 곳을 수리하게 됩니다.

## 4.2 예측·실제·확률을 한 표로

**패턴** 결과 표 만들고 정렬하기 그대로 정비팀에 넘길 목록

**정의** 예측·실제·확률 세 가지를 한 DataFrame 으로 묶고 확률 순으로 정렬하면 그 표가 곧 점검 우선순위 목록이 됩니다.

**형태** `pd.DataFrame({...}).sort_values("공고장확률", ascending=False)`

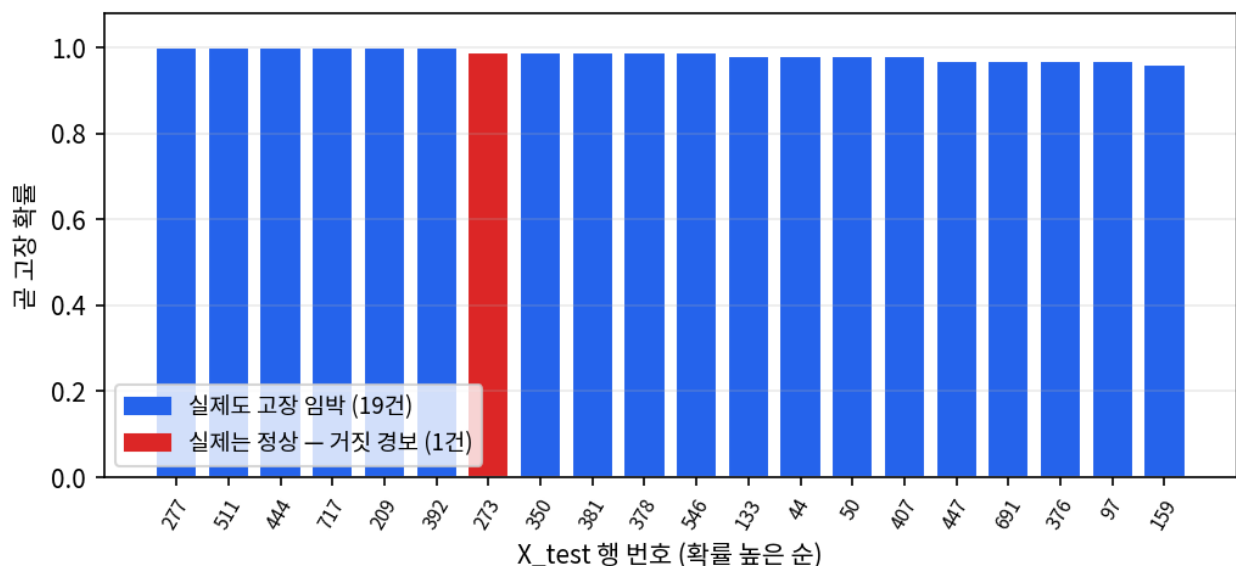
세 개를 따로 보면 대조하기 번거롭습니다. 한 표에 모으면 한 줄씩 눈으로 맞춰 볼 수 있습니다.

```
result = pd.DataFrame({
    "예측": y_pred,
    "실제": y_test.values,      # .values 로 인덱스를 떼고 순서만 맞춤
    "공고장확률": proba_1,
})

print("1로 예측된 행:", int((result["예측"] == 1).sum()), "건")
print(result.sort_values("공고장확률", ascending=False).head(8).to_string())
```

```
▶ 1로 예측된 행: 114 건
▶   예측 실제 공고장확률
▶ 277    1    1    1.00
▶ 511    1    1    1.00
▶ 444    1    1    1.00
▶ 717    1    1    1.00
▶ 209    1    1    1.00
▶ 392    1    1    1.00
▶ 273    1    0    0.99
▶ 350    1    1    0.99
```

맨 위 여섯 건은 **100 그루가 만장일치**로 곧 고장이라고 한 행이고, 전부 실제로도 고장 임박이었습니다. 일곱 번째 273 번만 실제로는 정상인데 99 그루가 고장이라고 봤습니다 — 확신에 찬 거짓 경보입니다.



▲ 확률 상위 20 건 — 파란 막대 19 건은 실제로도 고장 임박이었고, 빨간 막대 1 건(273 번)만 거짓 경보입니다

**왜 쓰나 정렬된 표 자체가 결과물입니다.** 723 행을 다 보라는 말은 아무도 실행할 수 없지만, 확률 순으로 정렬된 상위 20 건은 위에서부터 처리하면 되는 지시가 됩니다. 그중 19 건이 진짜라면 정비팀이 신뢰할 만합니다.

#### 주의

`y_test.values`에서 `.values`를 쓰는 이유는 **인덱스 때문**입니다. `y_test`는 원래 데이터의 인덱스(1431, 153, 514...)를 그대로 달고 있고 `y_pred`는 0부터 시작하는 배열입니다. 위처럼 한 번에 만들면 pandas가 알아서 맞춰 주지만, **표를 먼저 만들고 나중에 `result["실제"] = y_test`로 대입하면 인덱스가 안 맞아 723행 중 599행이 결측이 됩니다. 오류는 나지 않으니 표를 꼭 눈으로 확인하십시오.**

### 4.3 데이터만 바뀌면 그대로 쓸 수 있다

엔진 데이터로 끝낸 흐름을 소리 데이터에 그대로 옮겨 봅니다. 어제 잠깐 스쳤던 `27_mimii_features_sample.csv`, 1,000행짜리 소리 특징 데이터입니다.

```
mdf = pd.read_csv("../Data/27_mimii_features_sample.csv", encoding="utf-8")

mX = mdf[["rms", "spectral_centroid", "zero_crossing_rate"]] # 입력 — 소리 특징 3개
my = mdf["label"] # 정답 — 정상 0 / 이상 1

mX_train, mX_test, my_train, my_test = train_test_split(
    mX, my, test_size=0.2, random_state=42, stratify=my)

mi_model = RandomForestClassifier(n_estimators=100, random_state=42)
mi_model.fit(mX_train, my_train)
mi_pred = mi_model.predict(mX_test)

print(mi_pred[:20])

▶ [0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0]
```

**코드가 엔진 때와 거의 같습니다.** 파일 이름과 컬럼 이름만 바뀌었을 뿐, `train_test_split` → `RandomForestClassifier` → `fit` → `predict` 순서는 그대로입니다. 입력이 센서 6개에서 소리 특징 3개로 줄었는데도 손댈 곳이 없습니다. 참고로 이 소리 데이터에서는 정확도가 **99.5%**까지 나옵니다 — 엔진 데이터(93.6%)보다 훨씬 높는데, 문제가 더 쉬웠기 때문이지 모델이 더 좋아진 것은 아닙니다.

#### 개념

이것이 분류라는 문제 틀의 힘입니다. **데이터가 무엇이든 X와 y로 정리되기만 하면 같은 흐름이 돌아갑니다.** 진동이든 온도가든 소리가든, 앞단에서 X/y를 제대로 만드는 일이 끝나면 뒷단은 거의 그대로입니다. 37일차에 데이터 준비를 여섯 장에 걸쳐 다룬 이유가 여기서 다시 확인됩니다.

### 4.4 중요도가 낮은 센서를 빼면 어떻게 되나

중요도 표에서 `sensor_3`은 0.0658로 꼴찌였습니다. 그러면 아예 빼도 되는 걸까요. **빼고 다시 학습시켜 보면 바로 답이 나옵니다.**

```

조합 = {
    "센서 6 개 (전부)": feats,
    "sensor_3 제외 5 개": [f for f in feats if f != "sensor_3"],
    "상위 3 개만": ["sensor_7", "sensor_15", "sensor_2"],
    "상위 2 개만": ["sensor_7", "sensor_15"],
}

for 이름, 컬럼 in 조합.items():
    a, b, c, d = train_test_split(df[컬럼], y, test_size=0.2, random_state=42, stratify=y)
    m = RandomForestClassifier(n_estimators=100, random_state=42).fit(a, c)
    print(f"{이름:20s} 센서 {len(컬럼)}개 정확도 {accuracy_score(d, m.predict(b)):.4f}")

```

- ▶ 센서 6 개 (전부)      센서 6 개 정확도 0.9364
- ▶ sensor\_3 제외 5 개    센서 5 개 정확도 0.9378
- ▶ 상위 3 개만      센서 3 개 정확도 0.9212
- ▶ 상위 2 개만      센서 2 개 정확도 0.9101

가장 눈에 띄는 줄은 두 번째입니다. **꼴찌 센서를 빼자 정확도가 오히려 조금 올랐습니다**(0.9364 → 0.9378). 기여가 거의 없는 입력은 도움이 안 될 뿐 아니라 **약간의 잡음으로 작용**할 수 있다는 뜻입니다.

| 입력 구성              | 센서 수 | 정확도    | 전부 대비   | 읽는 법      |
|--------------------|------|--------|---------|-----------|
| 센서 6 개 (전부)        | 6    | 0.9364 | —       | 기준        |
| <b>sensor_3 제외</b> | 5    | 0.9378 | +0.14%p | 빼도 손해가 없다 |
| 상위 3 개만            | 3    | 0.9212 | -1.52%p | 조금씩 깎인다   |
| 상위 2 개만            | 2    | 0.9101 | -2.63%p | 확실히 나빠진다  |

그렇다고 중요도 낮은 것을 계속 빼도 되는 것은 아닙니다. 상위 2 개만 남기면 **2.6%p 가 낮아집니다**. 중요도 0.13 인 sensor\_4 나 0.15 인 sensor\_11 은 1 등만큼은 아니어도 **제 몫을 하고 있었다**는 뜻입니다. 중요도가 낮다는 것은 없어도 된다가 아니라 **혼자서는 결정적이지 않다**는 의미에 가깝습니다.

#### 설비 적용

이 실험이 현장에서는 **센서 예산 판단**이 됩니다. 센서 하나를 늘리면 설치비와 배선과 유지보수가 따라붙습니다. 중요도가 계속 바닥이고 빼도 성능이 유지되는 센서라면 새 라인에 굳이 달지 않는 근거가 됩니다. 반대로 **빼면 성능이 눈에 띄게 떨어지는 센서는 고장 나면 즉시 교체해야 할 우선순위**가 됩니다.

#### 주의

0.9364 와 0.9378 의 차이는 723 건 중 **한 건**입니다. 이 정도 차이를 성능이 올랐다고 단정하면 안 됩니다. random\_state 를 몇 개 바꿔 가며 평균을 내야 진짜 차이인지 알 수 있습니다. 여기서 확실히 말할 수 있는 것은 **오르지도 내리지도 않았**다는 것, 즉 빼도 손해가 없다는 사실까지입니다.

## 4 장 한 장 정리

| 배운 것                  | 한 줄 정의                      | 기억할 숫자                  |
|-----------------------|-----------------------------|-------------------------|
| .feature_importances_ | 어떤 입력이 판단에 많이 쓰였는지 비율로 알려준다 | 합이 1.0                  |
| 1 위 센서                | sensor_7 — 트리가 가장 먼저 묻는 값   | 중요도 0.287               |
| 우선순위 표                | 예측·실제·확률을 묶어 확률 순으로 정렬      | 상위 20 건 중 19 건 적중       |
| 센서 빼기                 | 꼴찌 센서는 빼도 성능이 유지된다          | 6 개 0.9364 / 5 개 0.9378 |
| 다른 데이터                | 컬럼 이름만 바꾸면 같은 흐름이 돈다        | 소리 데이터 0.995            |

#### 한 장 정리

4 장의 결론은 **모델이 왜 그렇게 판단했는지 물어볼 수 있다**는 것입니다. 중요도는 근거를 대는 수단이고, 정렬된 표는 그대로 지시서가 됩니다. 다만 중요도가 낮다는 말은 없어도 된다가 아니라 **혼자서는 결정적이지 않다**는 뜻입니다.

여기서 수업의 방향이 꺾입니다. 지금까지는 모델을 만드는 이야기였고, 지금부터는 **그 모델을 채점하는 이야기**입니다. 강사의 표현대로 "제목만 봐도 부정적인 얘기" — 그동안 배운 것의 한계를 짚는 시간입니다.

평가가 필요한 이유는 단순합니다. **채점 없이 현장에 붙이면 잘 잡는지 헛경보를 내는지 모르는 채로 설비를 맡기는 셈**이기 때문입니다. 면허 없는 사람에게 큰 트럭을 맡기지 않듯, 채점하지 않은 모델에 설비를 맡길 수는 없습니다.

## 5.1 정확도 — 가장 먼저 떠오르는 점수

**함수** `accuracy_score()` 맞힌 비율 계산

**정의** 맞힌 개수 ÷ 전체 개수입니다. 0 부터 1 사이의 값으로 나오고 보통 백분율로 읽습니다.

**형태** `accuracy_score(y_test, y_pred)`

인자 순서가 **정답이 먼저, 예측이 나중**입니다. 앞으로 배울 모든 평가 지표 함수가 이 순서를 따릅니다. 이번 장은 1,000 행짜리 `28_cmapss_fd001_sample.csv` 로 진행합니다.

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

cdf = pd.read_csv("../Data/28_cmapss_fd001_sample.csv", encoding="utf-8")
feats = ["sensor_2", "sensor_3", "sensor_4", "sensor_7", "sensor_11", "sensor_15"]
cX, cy = cdf[feats], cdf["failure_soon"]

cX_train, cX_test, cy_train, cy_test = train_test_split(
    cX, cy, test_size=0.2, random_state=42, stratify=cy)

c_model = RandomForestClassifier(n_estimators=100, random_state=42).fit(cX_train, cy_train)
cy_pred = c_model.predict(cX_test)

print("정확도:", accuracy_score(cy_test, cy_pred))
```

▶ 정확도: 0.865

**86.5%**입니다. 200 건 중 173 건을 맞혔습니다. 언뜻 보면 나쁘지 않은 점수입니다.

**왜 쓰나** 정확도는 **정상과 고장을 똑같이 한 개씩 세어** 맞힌 비율을 냅니다. 계산이 단순하고 직관적이라 가장 먼저 떠올리게 되는 지표입니다.

**응용** 정의를 손으로 확인해 두면 이 숫자가 어떻게 만들어지는지 몸에 남습니다. 같은 자리의 값이 같은 개수를 세어 전체로 나누면 됩니다.

```
correct = (cy_test == cy_pred).sum()
total = len(cy_test)

print("직접 계산:", correct / total)
print("함수 결과:", accuracy_score(cy_test, cy_pred))
```

▶ 직접 계산: 0.865

▶ 함수 결과: 0.865

두 값이 정확히 일치합니다. 정확도가 특별한 계산이 아니라 **맞은 개수를 세어 나눈 것**임을 확인한 셈입니다.

## 5.2 그런데 이 데이터는 기울어져 있다

점수를 해석하기 전에 데이터가 어떤 모양인지부터 봐야 합니다. 37 일차에 배운 **클래스 불균형**이 여기서 다시 나옵니다.

```
print(cy.value_counts())
print(cy.value_counts(normalize=True))
```

---

```
▶ failure_soon
▶ 0      781
▶ 1      219
▶ Name: count, dtype: int64
▶ failure_soon
▶ 0      0.781
▶ 1      0.219
▶ Name: proportion, dtype: float64
```

**정상 781 건, 고장 임박 219 건**입니다. 정상이 78.1%를 차지합니다. 이것은 데이터가 잘못된 것이 아닙니다. 설비는 **수명의 대부분을 멀쩡히 가동**하고 위험한 구간은 끝자락에 잠깐 몰려 있으니, 고장 데이터가 불균형한 것은 지극히 자연스럽습니다. 200 사이클을 사는 엔진에서 RUL 30 이하 구간은 15%뿐입니다.

### 개념

**불균형 자체는 결함이 아닙니다. 없애야 할 대상도 아닙니다.** 진짜 문제는 불균형한 데이터를 **정확도 같은 부적합한 지표로 평가하는 일**입니다. 게다가 failure\_soon의 기준을 30 보다 짧게 잡을수록 더 불균형해지므로, 37 일차에 정한 임계값이 오늘의 평가에까지 영향을 줍니다.

## 5.3 함정 — 아무것도 안 해도 78 점

이제 함정을 직접 만들어 봅시다. **센서를 보지도 않고 무조건 정상이라고만 찍는 모델**을 만들면 몇 점이 나올까요.

**함수** DummyClassifier 아무것도 학습하지 않는 기준선

**정의** 데이터를 보지 않고 정해진 전략대로만 답하는 모델입니다. strategy="most\_frequent"는 **항상 가장 많은 클래스로만 예측**합니다.

**형태** DummyClassifier(strategy="most\_frequent")

학습 없이 만들 수 있는 가장 단순한 비교 대상이라 **베이스라인**이라고 부릅니다.

```
from sklearn.dummy import DummyClassifier
import numpy as np

dummy = DummyClassifier(strategy="most_frequent")
dummy.fit(cX_train, cy_train)
print("더미 베이스라인:", dummy.score(cX_test, cy_test))
```

---

```
# 같은 일을 직접 해 보기 — 전부 0 으로 채운 배열과 정답을 대조
all_normal = np.zeros(len(cy_test))
print("무조건 정상 :", accuracy_score(cy_test, all_normal))
print("잡아낸 고장 :", int(((all_normal == 1) & (cy_test == 1)).sum()), "건")
```

---

```
▶ 더미 베이스라인: 0.78
▶ 무조건 정상 : 0.78
▶ 잡아낸 고장 : 0 건
```

**78 점입니다. 그런데 잡아낸 고장은 0 건입니다.** 센서를 한 번도 들여다보지 않은 모델이, 정작 찾아내려던 것을 하나도 못 찾고도 78 점을 받았습니다. 점수가 78 점이라는 사실과 이 모델이 아무 쓸모도 없다는 사실이 동시에 참입니다.



**왜 쓰나** 이것이 **정확도의 함정**입니다. 불균형이 심할수록 더 심해집니다. 정상이 95%인 데이터라면 무조건 정상 모델의 정확도는 95%가 됩니다.

#### 자주 하는 실수

함정이 위험한 진짜 이유는 **점수가 나빠 보이지 않는** 데 있습니다. 30%가 나왔다면 누구나 이상하다며 다시 봤을 텐데, 95%가 나오면 사람은 자연스럽게 안심하고 넘어갑니다. **높은 점수가 오히려 위험을 가립니다**. 불균형 데이터에서 정확도가 높게 나올수록 한 번 더 의심해야 합니다.

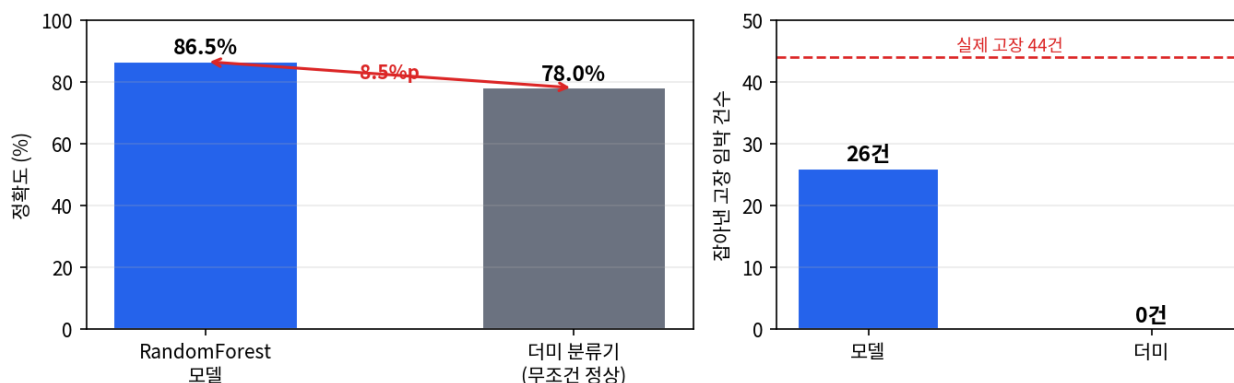
## 5.4 베이스라인과 비교해야 해석이 된다

그러면 우리 모델의 86.5%는 잘한 것일까요. **이 질문은 혼자서는 답할 수 없습니다**. 비교 대상이 있어야 합니다.

```
model_acc = accuracy_score(cy_test, cy_pred)
dummy_acc = dummy.score(cX_test, cy_test)

print("우리 모델 :", model_acc)
print("베이스라인:", dummy_acc)
print("차이(%p) :", round((model_acc - dummy_acc) * 100, 2))
```

- ▶ 우리 모델 : 0.865
- ▶ 베이스라인: 0.78
- ▶ 차이(%p) : 8.5



▲ 왼쪽 — 정확도는 8.5%p 차이뿐입니다. 오른쪽 — 그런데 잡아낸 고장은 26 건과 0 건으로 갈립니다. 두 그림을 같이 봐야 모델의 값어치가 보입니다

**8.5%p 차이입니다**. 교안이 제시한 기준으로 읽으면 1~2%p는 학습이 거의 안 된 것, 10%p 이상이면 의미 있는 학습입니다. 8.5%p는 그 사이에 있으니 **학습은 분명히 됐지만 압도적이지는 않은** 상태입니다.

그런데 오른쪽 그림이 더 중요합니다. 정확도로는 8.5%p 차이인데, **잡아낸 고장은 26 건 대 0 건**입니다. 정확도라는 지표가 이 차이를 거의 반영하지 못한다는 뜻입니다. 실제 고장 44 건 중 26 건을 잡았으니 여전히 18 건을 놓쳤지만, 0 건과 26 건의 차이는 현장에서 결코 8.5%p 짜리 차이가 아닙니다.

| 비교 항목  | RandomForest 모델 | 더미 (무조건 정상) | 차이    |
|--------|-----------------|-------------|-------|
| 정확도    | 86.5%           | 78.0%       | 8.5%p |
| 잡아낸 고장 | 26 건            | 0 건         | 26 건  |
| 놓친 고장  | 18 건            | 44 건        | —     |
| 학습 여부  | 센서 6 개를 보고 판단   | 데이터를 보지 않음  | —     |

#### 개념

오늘의 결론은 이 한 문장입니다. **정확도 하나만 보면 고장을 다 놓친 모델도 좋아 보인다**. 그래서 정확도 옆에는 항상 기준선이 필요하고, 기준선만으로도 부족합니다. 정상과 고장을 갈라서 봐야 합니다 — **그것이 다음 시간에 배울 혼동행렬**입니다.

## 연결

1~4장에서 만든 엔진 모델의 정확도는 93.6%였습니다. 같은 방식으로 재 보면 그 데이터의 베이스라인은 **82.9%**이고, 차이는 **10.8%p**입니다. 1,000행 데이터(8.5%p)보다 낮습니다. 데이터가 3.6배 많아 학습 재료가 넉넉했던 것이 이유로 보입니다. 점수는 데이터마다 다르게 읽어야 하고, 그래서 비교 대상 없이 숫자만 옮겨 적으면 안 됩니다.

## 5장 한 장 정리

| 배운 것                                        | 한 줄 정의                       | 기억할 숫자              |
|---------------------------------------------|------------------------------|---------------------|
| <code>accuracy_score(y_test, y_pred)</code> | 맞힌 개수 ÷ 전체 — 인자 순서는 (정답, 예측) | 엔진 1,000행 0.865     |
| 클래스 불균형                                     | 한쪽 답이 훨씬 많아 점수가 부풀려지는 상태     | 정상 78.1% · 고장 21.9% |
| <code>DummyClassifier</code>                | 무조건 다수 쪽만 찍는 기준선 모델          | 정확도 0.780           |
| 의미 있는 차이                                    | 모델 점수 - 베이스라인 점수             | 8.5%p               |
| 진짜 성과                                       | 더미가 잡은 고장은 0건                | 모델은 44건 중 26건       |

## 한 장 정리

5장의 결론은 **점수는 혼자서 아무 뜻도 없다**는 것입니다. 86.5%가 좋은 점수인지는 78.0%라는 기준선을 알아야 판단할 수 있습니다. 그리고 8.5%p라는 숫자보다 더 중요한 사실은 **더미가 고장을 단 한 건도 잡지 못했다**는 것입니다 — 그 이야기를 다음 시간에 이어 갑니다.

복습할 때는 이 표만 훑어도 됩니다. 기억이 안 나는 줄이 있으면 그 도구가 나온 장으로 돌아가십시오.

### A.1 모델 만들기부터 예측까지 (27\_03)

| 도구         | 쓰는 법                                                                   | 하는 일                                    |
|------------|------------------------------------------------------------------------|-----------------------------------------|
| <b>함수</b>  | <code>RandomForestClassifier(n_estimators=100, random_state=42)</code> | 트리 100 그루짜리 분류기를 만듭니다. 아직 빈 상자입니다.      |
| <b>메서드</b> | <code>model.fit(X_train, y_train)</code>                               | 학습용만 넣어 트리 100 그루를 동시에 학습시킵니다.          |
| <b>메서드</b> | <code>model.predict(X_test)</code>                                     | 각 행에 0 또는 1 판단을 돌려줍니다.                  |
| <b>메서드</b> | <code>model.predict_proba(X_test)</code>                               | [0 일 확률, 1 일 확률]을 돌려줍니다. 두 값의 합은 1 입니다. |
| <b>문법</b>  | <code>proba[:, 1]</code>                                               | 모든 행의 두 번째 열, 즉 곧 고장일 확률만 꺼냅니다.         |
| <b>속성</b>  | <code>model.feature_importances_</code>                                | 각 입력이 판단에 쓰인 정도. 괄호를 붙이지 않습니다.          |
| <b>패턴</b>  | <code>pd.Series(중요도, index=feats).sort_values(ascending=False)</code>  | 중요도에 센서 이름을 붙여 큰 순으로 정렬합니다.             |
| <b>패턴</b>  | <code>pd.DataFrame({...}).sort_values("곧고장확률", ascending=False)</code> | 예측·실제·확률을 묶어 점검 우선순위 목록을 만듭니다.          |

### A.2 채점하기 (28\_01 PART 01)

| 도구         | 쓰는 법                                                   | 하는 일                            |
|------------|--------------------------------------------------------|---------------------------------|
| <b>함수</b>  | <code>accuracy_score(y_test, y_pred)</code>            | 맞힌 비율. 정답이 먼저, 예측이 나중입니다.       |
| <b>패턴</b>  | <code>(y_test == y_pred).sum() / len(y_test)</code>    | 정확도를 손으로 계산합니다. 함수 결과와 같아야 합니다. |
| <b>함수</b>  | <code>DummyClassifier(strategy="most_frequent")</code> | 항상 다수 클래스로만 찍는 기준선 모델입니다.       |
| <b>메서드</b> | <code>dummy.score(X_test, y_test)</code>               | 베이스라인 정확도를 바로 돌려줍니다.            |
| <b>함수</b>  | <code>np.zeros(len(y_test))</code>                     | 0 으로만 채운 배열. 무조건 정상 예측을 흉내 냅니다. |
| <b>메서드</b> | <code>y.value_counts(normalize=True)</code>            | 클래스 비율을 확인해 불균형 정도를 봅니다.        |

### A.3 오늘의 숫자

| 데이터        | 항목                                                   | 값                                                          |
|------------|------------------------------------------------------|------------------------------------------------------------|
| 엔진 3,614 행 | 모델 정확도 · 평가용 행 수                                     | 93.6% · 723 행                                              |
|            | <code>n_estimators</code> 1 / 50 / 100 / 500 일 때 정확도 | 89.2% / 93.6% / 93.6% / 93.9%                              |
|            | 중요도 1~2 위                                            | <code>sensor_7</code> 0.287 · <code>sensor_15</code> 0.200 |
|            | 확률 0.9 이상 · 그중 실제 고장                                 | 39 건 · 35 건 (89.7%)                                        |
|            | 확률 0.4~0.6 · 그중 실제 고장                                | 38 건 · 22 건 (57.9%)                                        |
|            | <code>random_state</code> 42 vs 7 판정 불일치             | 723 건 중 14 건                                               |

| 데이터        | 항목                     | 값                               |
|------------|------------------------|---------------------------------|
|            | 1 로 예측된 행 · 상위 20 건 적중 | 114 건 · 19/20                   |
| 소리 1,000 행 | MIMII 정확도              | 99.5%                           |
| 엔진 1,000 행 | 클래스 분포                 | 정상 781 · 고장 219 (78.1% / 21.9%) |
|            | 모델 · 더미 · 차이           | 86.5% · 78.0% · 8.5%p           |
|            | 잡아낸 고장 (모델 / 더미)       | 26 건 / 0 건 · 실제 고장 44 건         |

## B.1 오늘 코드에서 틀리기 쉬운 곳

| ✗ 이렇게 하면                                           | 무슨 일이                                          | ✓ 바로잡기                                      |
|----------------------------------------------------|------------------------------------------------|---------------------------------------------|
| <code>model.fit(X_test, y_test)</code>             | 시험 문제로 공부한 셈이라 평가가 의미를 잃습니다.                   | <code>model.fit(X_train, y_train)</code>    |
| <code>proba[:1]</code>                             | 첫 번째 행 하나만 나옵니다. 심표 하나 차이입니다.                  | <code>proba[:, 1]</code>                    |
| <code>model.feature_importances_()</code>          | 속성에 괄호를 붙이면 <code>TypeError</code> 가 납니다.      | <code>model.feature_importances_</code>     |
| <code>result["실제"] = y_test</code>                 | 인덱스가 안 맞아 723 행 중 599 행이 결측이 됩니다. 오류는 안 납니다.   | <code>result["실제"] = y_test.values</code>   |
| <code>accuracy_score(y_pred, y_test)</code>        | 우연히 값은 같지만 인자 순서 관습이 깨집니다. 다른 지표에서는 결과가 달라집니다. | <code>accuracy_score(y_test, y_pred)</code> |
| <code>pd.Series(model.feature_importances_)</code> | 센서 이름 없이 0,1,2... 로만 나와 어느 센서인지 알 수 없습니다.      | <code>pd.Series(..., index=feats)</code>    |
| 정확도만 보고 모델을 판단                                     | 무조건 정상 모델도 78 점이라 좋아 보입니다.                     | 베이스라인과 나란히 놓고 비교                            |
| 중요도 1위를 고장 원인으로 단정                                 | 판단에 많이 쓰였다는 뜻일 뿐입니다.                           | 단서로 쓰고 설비 전문가가 확인                           |

## B.2 실습 코드에서 잡아낸 조용한 오류

오늘 실습 노트북의 마지막 부분에 **오류 메시지 없이 결과만 틀리는** 줄이 하나 있었습니다. 실행해 보지 않으면 절대 못 잡는 유형이라 따로 정리해 둡니다.

```
# ✗ 실습 노트북에 적힌 코드
print("무조건 정상 데이터일 때의 정확도는?:", accuracy_score(cy_pred, all_normal))

# ✓ 교안이 제시한 코드
print("무조건 정상:", accuracy_score(cy_test, all_normal))
```

차이는 첫 번째 인자입니다. 정답인 `cy_test` 자리에 **모델의 예측인** `cy_pred`가 들어갔습니다. 그러면 정답과 대조하는 것이 아니라 모델의 예측 중 몇 개가 0 인지를 세게 됩니다. 전혀 다른 값이 나옵니다.

```
print("정답 vs 무조건정상:", accuracy_score(cy_test, all_normal)) # 진짜 베이스라인
print("예측 vs 무조건정상:", accuracy_score(cy_pred, all_normal)) # 예측 중 0의 비율
```

▶ 정답 vs 무조건정상 : 0.78  
▶ 예측 vs 무조건정상 : 0.825

0.78과 0.825, **4.5%p 차이**입니다. 그런데 이 값이 그대로 다음 계산에 들어가면서 결론까지 바뀝니다.

| 계산       | 베이스라인 | 모델과의 차이 | 교안 기준으로 읽으면          |
|----------|-------|---------|----------------------|
| ✗ 잘못된 코드 | 0.825 | 4.0%p   | 1~2%p와 10%p 사이 — 애매  |
| ✓ 올바른 코드 | 0.780 | 8.5%p   | 10%p에 가까움 — 의미 있는 학습 |

같은 데이터, 같은 모델인데 인자 하나를 바꿔 쓴 것만으로 **학습이 애매하다**에서 **학습이 의미 있다**로 결론이 넘어갑니다.

`DummyClassifier`가 돌려준 0.78과 손으로 계산한 값이 다르면 둘 중 하나가 틀린 것이니, **두 방법으로 구해 대조하는 습관**이 이런 오류를 잡아 줍니다.

#### 주의

평가 지표 함수는 거의 다 (정답, 예측) 순서입니다. `accuracy_score` 는 두 배열이 같은지만 세기 때문에 순서를 바꿔도 값이 우연히 같지만, 다음 시간에 배울 **정밀도·재현율**은 순서를 바꾸면 값이 완전히 달라집니다. 지금부터 순서를 지키는 습관을 들여야 합니다.

## APPENDIX C

## 실습 하이라이트 &amp; 다음 시간

오늘 손으로 짚은 것 · 다음에 할 것

## C.1 27\_03 실습 흐름

| #  | 실습                     | 확인할 것                                             |
|----|------------------------|---------------------------------------------------|
| 1  | 모델 생성과 학습              | 출력에 RandomForestClassifier(random_state=42)가 나오는가 |
| 2  | 예측 실행과 개수 확인           | 예측 개수와 평가용 행 수가 둘 다 723 인가                        |
| 3  | 예측·실제 앞 10 개 비교        | 같은 순서로 나란히 놓이는가                                   |
| 4  | predict_proba 확률 받기    | 두 열을 더하면 1 이 되는가                                  |
| 5  | 1 일 확률만 꺼내기            | proba[:, 1] 의 심표를 빠뜨리지 않았는가                       |
| 6  | random_state 7 로 바꿔 학습 | 앞 10 개는 같고 전체로는 14 건이 달라지는가                       |
| 7  | 같은 값으로 재현              | 42 로 다시 돌리면 완전히 같은가                               |
| 8  | 센서 중요도 정렬              | 합이 1 이 되는가 · 이름이 함께 나오는가                          |
| 9  | 결과 표 만들기               | 예측·실제·확률 세 열이 같은 행을 가리키는가                         |
| 10 | 우선순위 정렬                | 상위 15 건 중 실제 고장이 14 건인가                           |
| 11 | MIMII 소리 데이터에 적용       | 컬럼 이름만 바뀌도 흐름이 그대로인가                              |

## C.2 28\_01 실습 흐름 (PART 01 까지)

| # | 실습                 | 확인할 것                                 |
|---|--------------------|---------------------------------------|
| 1 | 1,000 행 데이터로 학습·예측 | 정확도 0.865 가 나오는가                      |
| 2 | 정확도 손계산과 함수 대조     | 두 값이 정확히 같은가                          |
| 3 | 클래스 비율 확인          | 정상 781 · 고장 219 — 얼마나 기울어졌는가          |
| 4 | 더미 분류기 베이스라인       | 0.78 — 센서를 안 보고도 이만큼 나오는가             |
| 5 | 무조건 정상 예측의 정확도     | 인자 순서가 (cy_test, all_normal)인가 (부록 B) |
| 6 | 두 정확도 차이 계산        | 8.5%p — 의미 있는 수준인지 판단할 수 있는가          |

## C.3 다음 시간 예고 — 혼동행렬과 정밀도·재현율

강사가 오늘 혼동행렬을 열었다가 "무리해서 들어가지 않고 다음 주 월요일에 하겠다"며 접었습니다. 오늘 배운 것이 바로 그 필요성이었으니, 다음 시간은 자연스럽게 이어집니다.

| 단계   | 도구                                            | 하는 일                                            |
|------|-----------------------------------------------|-------------------------------------------------|
| 혼동행렬 | <code>confusion_matrix(y_test, y_pred)</code> | 정답과 예측을 네 칸으로 갈라 셉니다.<br>TP·FN·FP·TN 입니다.       |
| 펼치기  | <code>.ravel()</code>                         | 네 값을 순서대로 꺼냅니다. 순서가 tn, fp, fn, tp 라 헛갈리기 쉽습니다. |
| 시각화  | <code>ConfusionMatrixDisplay</code>           | 네 칸을 색으로 칠해 어느 칸이 두꺼운지 한눈에 봅니다.                 |
| 정밀도  | <code>precision_score</code>                  | 1 이라고 답한 것 중 몇 개가 맞았나 — 헛걸음의 지표입니다.             |
| 재현율  | <code>recall_score</code>                     | 실제 1 중 몇 개를 잡았나 — 놓침의 지표입니다.                    |

| 단계 | 도구                | 하는 일                      |
|----|-------------------|---------------------------|
| 확장 | 28_02 고장 예측 결과 해석 | 예측을 정비 판단으로 옮기는 마무리 장입니다. |

#### 연결

오늘 배운 것이 다음 시간의 출발점이 됩니다. **더미 모델이 잡은 고장이 0 건**이라는 사실은 재현율이 0%라는 말과 같고, **우리 모델이 44 건 중 26 건을 잡았다**는 것은 재현율 59.1%라는 뜻입니다. 정확도로는 8.5%p 차이였던 것이 재현율로 보면 0%와 59.1%의 차이가 됩니다. 지표를 바꾸면 같은 결과가 전혀 다르게 보인다는 것이 다음 시간의 핵심입니다.



오늘 처음으로 모델을 학습시키고 예측을 받아 봤습니다. 지난 여섯 장에 걸쳐 데이터를 준비한 덕분에 학습은 세 줄로 끝났습니다. 그리고 마지막에 그 결과를 의심하는 법까지 배웠습니다. 모델을 만드는 것보다 **그 모델이 쓸 만한지 판단하는 것**이 더 어렵다는 사실을 오늘 확인한 셈입니다.

### 오늘 확실히 하고 넘어갈 것 다섯 가지

| # | 확인 질문                                                        | 대답이 안 나오면 |
|---|--------------------------------------------------------------|-----------|
| 1 | 트리 한 그루 대신 숲을 쓰는 이유를 설명할 수 있는가                               | 1 장 1.2   |
| 2 | <code>n_estimators</code> 를 500 이 아니라 100 으로 두는 이유를 말할 수 있는가 | 1 장 1.3   |
| 3 | 확률 0.95 인 예측과 0.52 인 예측을 현장에서 다르게 다루는 이유는                    | 3 장 3.2   |
| 4 | 중요도 1 위 센서를 고장 원인이라고 말하면 안 되는 이유는                            | 4 장 4.1   |
| 5 | 정확도 86.5%가 좋은 점수인지 판단하려면 무엇이 더 필요한가                          | 5 장 5.4   |

### 복습 루틴 제안

**오늘 저녁 20 분** — 부록 A 치트시트를 훑고 위 다섯 질문에 소리 내어 답해 보십시오. 막히는 항목만 해당 절로 돌아가면 됩니다.

**주말 한 시간** — 27\_03 실습을 `n_estimators` 만 10 으로 바꿔 처음부터 다시 돌려 보십시오. 정확도가 얼마나 떨어지는지, 중요도 순위가 바뀌는지, 확률 분포가 어떻게 성글어지는지 보면 1 장이 몸에 납니다.

**다음 수업 전 10 분** — 부록 C 의 예고 표를 읽어 두십시오. TP·FN·FP·TN 네 글자가 미리 눈에 익으면 수업 중에 코드를 따라 치는 여유가 생깁니다.

#### 설비 적용

오늘 배운 순서 — **모델을 만들고, 확신도로 순서를 매기고, 중요 센서로 점검 부위를 좁히고, 기준선과 비교해 값어치를 따진다** — 는 어떤 예지보전 프로젝트에서도 반복됩니다. 특히 마지막 단계를 건너뛰는 일이 흔한데, **비교 대상 없는 점수는 보고서에 적을 수 없는 숫자**라는 것을 오늘 확인했습니다.